

React: `useState`, eventi e stato

A cosa serve lo stato?

Una normale variabile JavaScript può cambiare, ma React non aggiorna automaticamente la pagina quando questa cambia. Per questo motivo, se vogliamo che un componente reagisca a un cambiamento di valore, dobbiamo usare **`useState`** (stato).

Lo **stato** è un dato che React tiene in memoria per un componente. Quando aggiorniamo lo stato, React esegue di nuovo il componente e aggiorna la parte di pagina che deve cambiare.

Esempi di stato:

- il numero mostrato da un contatore;
- una lampadina accesa o spenta;
- l'elemento aperto/chiuso di un accordion (UI a fisarmonica).

Usare `useState`

Prima importiamo l'Hook:

```
import { useState } from 'react';
```

Poi lo usiamo **dentro il componente, prima del `return`**:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return <p>{count}</p>;  
}
```

`useState(0)` restituisce due elementi, che destrutturiamo:

- **`count`**: il valore attuale dello stato;
- **`setCount`**: la funzione che useremo per richiederne l'aggiornamento;
- **`0`**: il valore iniziale, usato al primo render.

Il nome è a nostra scelta, ma si usa la convenzione **valore / setValore**. (**`state`** e **`setState`**)

Aggiornare lo stato

Non modifichiamo direttamente lo stato:

```
// ❌ Non fare
```

```
count = count + 1;
```

Usiamo invece la funzione `set`:

```
setCount(0); // assegna un valore preciso
```

Quando il nuovo valore dipende da quello precedente, passiamo una funzione:

```
setCount((previousCount) => previousCount + 1);
```

Questo è importante perché gli aggiornamenti dello stato vengono programmati da React e il valore disponibile nello stesso render non cambia immediatamente (ma solo al prossimo render)

Eventi in React

Gli eventi React si scrivono in **camelCase** e ricevono una funzione:

```
function Button() {  
  function handleClick() {  
    console.log('Click!');  
  }  
  
  return <button onClick={handleClick}>Premi</button>;  
}
```

Eventi comuni sono `onClick`, `onChange` e `onSubmit`. In JSX sono props speciali degli elementi.

Attenzione a non chiamare subito la funzione:

```
<button onClick={handleClick}>Corretto</button>  
<button onClick={() => setCount(0)}>Corretto con funzione inline</button>  
<button onClick={handleClick()}>Sbagliato: viene eseguita durante il  
render</button>
```

React gestisce per noi il collegamento degli eventi agli elementi e la relativa pulizia quando quegli elementi vengono rimossi.

Regole fondamentali degli Hook

`useState` è un **Hook**, cioè una funzione React il cui nome inizia con `use`.

Gli Hook:

- si chiamano dentro un componente React (o dentro un altro Hook personalizzato);

- si chiamano al livello principale del componente, prima di eventuali **return** anticipati;
- non si chiamano dentro **if**, cicli o funzioni annidate.

```
function Example({ isVisible }) {  
  const [count, setCount] = useState(0); // ✅ sempre nello stesso ordine  
  
  if (!isVisible) return null;  
  
  return <p>{count}</p>;  
}
```

React associa i diversi Hook in base all'ordine delle chiamate. Per questo tale ordine deve restare uguale a ogni render.

Dove mettere lo stato?

Metti lo stato nel componente più vicino che ne ha bisogno:

- se ogni componente deve essere indipendente, tieni lo stato al suo interno;
- se più componenti devono condividere o sincronizzare un valore, sposta lo stato nel parent comune e passalo tramite props.

Un figlio che riceve valore e comportamento dal parent è un **componente controllato**:

```
function App() {  
  const [count, setCount] = useState(0);  
  
  function handleIncrement() {  
    setCount((previousCount) => previousCount + 1);  
  }  
  
  return <Counter count={count} onIncrement={handleIncrement} />;  
}  
  
function Counter({ count, onIncrement }) {  
  return <button onClick={onIncrement}>Conteggio: {count}</button>;  
}
```

La prop **onIncrement** è una **callback prop**: il parent passa una funzione e il figlio la esegue in risposta a un evento. Nomi come **onIncrement**, **onSwitch** e **onToggle** rendono chiara l'intenzione e non espongono direttamente la funzione **set**.

Esercizi guidati

1. Contatore

Obiettivo: aumentare, diminuire e azzerare un numero.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount((previousCount) => previousCount + 1);
  }

  function reset() {
    setCount(0);
  }

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={reset}>Reset</button>
      <button onClick={increment}>+1</button>
    </div>
  );
}

export default Counter;
```

Passaggi:

1. `count` parte da `0`.
2. I pulsanti eseguono funzioni tramite `onClick`.
3. Incremento dipende dal valore precedente, quindi usa la forma con callback.
4. Il reset assegna direttamente `0`.

2. Lampadina on/off

Obiettivo: cambiare immagine e testo in base allo stato.

```
import { useState } from 'react';

function LightBulb() {
  const [isOn, setIsOn] = useState(false);

  function toggleLight() {
    setIsOn((previousIsOn) => !previousIsOn);
  }

  return (
    <div>
      <img
        src={isOn ? '/img/bulb-on.png' : '/img/bulb-off.png'}
        alt={isOn ? 'Lampadina accesa' : 'Lampadina spenta'}
      />
    </div>
  );
}
```

```

    <p>La lampadina è {isOn ? 'accesa' : 'spenta'}.</p>
    <button onClick={toggleLight}>{isOn ? 'Spegni' : 'Accendi'}</button>
  </div>
);
}

export default LightBulb;

```

Passaggi:

1. `isOn` è un booleano: `false` significa spenta, `true` accesa.
2. Il ternario sceglie immagine, testo e label del pulsante.
3. `!previousIsOn` inverte lo stato precedente.
4. Se renderizzi più `LightBulb`, ognuna avrà il proprio stato indipendente.

Sostituisci i percorsi delle immagini con quelli presenti nel tuo progetto.

3. Accordion controllato

Obiettivo: tenere aperto un solo elemento alla volta. Lo stato si trova nel parent e contiene l'`id` dell'elemento aperto oppure `null`.

```

import { useState } from 'react';

const accordionItems = [
  {
    id: 1,
    title: "Che cos'è React?",
    content: 'Una libreria per creare interfacce.',
  },
  {
    id: 2,
    title: "Che cos'è lo stato?",
    content: "Un dato che può aggiornare l'interfaccia.",
  },
  {
    id: 3,
    title: "Che cos'è una prop?",
    content: 'Un valore passato da parent a figlio.',
  },
];

function AccordionItem({ title, content, isOpen, onToggle }) {
  return (
    <section>
      <h2>
        <button onClick={onToggle}>{title}</button>
      </h2>

      {isOpen && <p>{content}</p>}
    </section>
  );
}

```

```
    </section>
  );
}

function AccordionList() {
  const [openItemId, setOpenItemId] = useState(null);

  function toggleItem(id) {
    setOpenItemId((previousId) => (previousId === id ? null : id));
  }

  return (
    <div>
      {accordionItems.map((item) => (
        <AccordionItem
          key={item.id}
          title={item.title}
          content={item.content}
          isOpen={openItemId === item.id}
          onToggle={() => toggleItem(item.id)}
        />
      ))}
    </div>
  );
}

export default AccordionList;
```

Passaggi:

1. Il parent conserva un solo `openItemId`, quindi può esserci un solo elemento aperto.
2. Con `map` creiamo un componente per ogni oggetto dell'array.
3. `isOpen={openItemId === item.id}` dice al figlio se mostrare il contenuto.
4. `onToggle` permette al figlio di chiedere al parent di aggiornare lo stato.
5. Se clicchiamo l'elemento già aperto, salviamo `null` e lo chiudiamo; altrimenti salviamo il nuovo `id`.

Riepilogo

- `useState` conserva un dato tra i render.
- La funzione `set` richiede un aggiornamento e provoca un nuovo render.
- Se il nuovo stato dipende dal precedente, usa `setState(previousState => ...)`.
- Gli eventi React sono props in camelCase che ricevono funzioni.
- Lo stato locale rende i componenti indipendenti; lo stato nel parent permette di sincronizzarli.
- Props e callback props collegano parent e figli mantenendo chiare le responsabilità.